Ders 16 Çalışma Özeti

Ders 16 Çalışma Özeti

Genel Konular

- Dersin tarihi 15 Kasım 2021 Pazartesi. Ders Zoom üzerinden yapıldı.
  - Asıl ders yaklaşık 00:08'de başlar. İlk dakikalar katılım beklemesidir.
  - Konu: Silberschatz, *Operating System Concepts* (9. baskı), Bölüm 4: İş Parçacıkları (Threads).
- Vize duyurusu ve sınav biçimi
- İş parçacığı kullanmanın motivasyonu
  - Süreç (process) oluşturmanın maliyeti ve iş parçacığının daha hafif olması.
  - Örnekler: metin düzenleyici, web sunucusu.
- Çok iş parçacıklı yapının faydaları
  - Duyarlılık (responsiveness), kaynak paylaşımı (resource sharing), ekonomi (economy), ölçeklenebilirlik (scalability).
- Çok çekirdekli programlama (multicore programming)
  - Veri paralelliği (data parallelism) ve görev paralelliği (task parallelism).
  - Zorluklar: veri bağımlılığı, test ve hata ayıklama.
  - Paralellik (parallelism) ile eşzamanlılık (concurrency) farkı.
  - Donanım iş parçacıkları (örneğin 4 çekirdekte 8 iş parçacığı).
- Tek ve çok iş parçacıklı süreçlerin yapısı
- Amdahl Yasası (Amdahl's Law)
- Kullanıcı seviyesi (user-level) ve çekirdek seviyesi (kernel-level) iş parçacıkları
- Çoklu iş parçacığı modelleri
  - Çoktan teke (many-to-one), teke tek (one-to-one), çoktan çoka (many-to-many), iki seviyeli (two-level).
- İş parçacığı kütüphaneleri
  - POSIX Pthreads, Windows iş parçacıkları, Java iş parçacıkları.
  - Pthreads toplama örneği, `pthread_create`, `pthread_join`.
- Örtük iş parçacığı kullanımı (implicit threading)
  - İş parçacığı havuzu (thread pool), OpenMP, Grand Central Dispatch.
- İş parçacığı kullanımındaki sorunlar
  - `fork()` ve `exec()` davranışı, sinyal işleme, iş parçacığı iptali (thread cancellation), iş parçacığına özel depolama (thread-local storage, TLS), zamanlayıcı aktivasyonları (scheduler activations) ve hafif süreç (lightweight process, LWP).

Hocanın Özellikle Vurguladığı Kısımlar

- Gelecek hafta vize haftası. Sınav programı ilan edildi.
- Vize soruları **çoktan çok seçmeli** olacak.
  - Bir soruda birden fazla doğru cevap olabilir. Bütün doğruları seçerseniz doğru sayılır.
  - Arada yanlış bir şık seçerseniz soru yanlış sayılır.
  - Yanlışlar doğruları götürür: 4 şıklı soruda 3 yanlış 1 doğruyu, 5 şıklı soruda 4 yanlış 1 doğruyu götürür.
  - Sorular varsayılan olarak İngilizce olacak.
- Laboratuvarda yapılan her şeyden sorumlusunuz.
  - Laboratuvarda doğrudan sınav yapılmıyor. Ama orada konuşulan pratik bilgi sınavda çıkabilir. Bu birinci vize, ikinci vize ve final için geçerli.
- Bölümün tamamından sorumlusunuz.

  - Derste anlatılmayan işletim sistemi örnekleri (slaytların son kısmı) da dahil.
- İş parçacıkları kodu, veriyi, yığın belleği (heap) ve açık dosyaları ortak kullanır.
  - Ortak alana erişimi düzenlemek uygulama geliştiricinin sorumluluğundadır. İşletim sistemi yalnızca mekanizmayı sağlar.
- İş parçacığı fonksiyonunun içinde `exit()` çağırmayın.
  - `exit()` bütün süreci sonlandırır. İş parçacığını bitirmek için `pthread_exit()` ya da `return` kullanılır.
- C'de `string` diye bir veri tipi yoktur. `argv[1]`'in tipi `char *`'dır.

Kısa Tekrar Notları

- Süreç kopyalamak (`fork`) kod, veri, yığın bellek ve yığın (stack) alanlarının hepsini kopyalar.
- İş parçacığı aynı süreç içinde yeni bir akıştır. Kendine ait yalnızca şunlar vardır:
  - iş parçacığı kimliği
  - program sayacı (program counter)
  - yazmaç kümesi (register set)
  - yığın (stack)
- Faydalar: duyarlılık, kaynak paylaşımı, ekonomi, ölçeklenebilirlik.
- Paralellik: birden fazla görev gerçekten aynı anda çalışır (farklı çekirdeklerde).
- Eşzamanlılık: tek çekirdekte zaman paylaşımıyla görevler aynı anda ilerliyormuş gibi görünür.
- Amdahl Yasası: hızlanma $\leq \dfrac{1}{S + \frac{1-S}{N}}$. S seri kısım, N çekirdek sayısı.
  - %25 seri, %75 paralel ve 2 çekirdek: $\frac{1}{0.25+0.375} = 1.6$.
  - $N \to \infty$ iken hızlanma $\frac{1}{S}$'ye yaklaşır. %25 seri kısım için en çok 4 kat.
- Çoktan teke: bir iş parçacığı bloklanınca bütün süreç bloklanır. Çok çekirdekten faydalanamaz.
- Teke tek: her kullanıcı iş parçacığına bir çekirdek iş parçacığı. Daha çok eşzamanlılık, ama daha çok maliyet. Bu yüzden işletim sistemleri iş parçacığı sayısını sınırlayabilir.
- Çoktan çoka: çok kullanıcı iş parçacığı, daha az ya da eşit sayıda çekirdek iş parçacığına eşlenir. Biri bloklanınca diğerleri çalışmaya devam eder.
- İki seviyeli: çoktan çoka + önemli bir iş parçacığını doğrudan bir çekirdek iş parçacığına bağlama.
- Pthreads kullanıcı ya da çekirdek seviyesinde gerçeklenebilir. Windows iş parçacıkları çekirdek seviyesindedir. Java iş parçacıkları JVM üzerinden işletim sisteminin kütüphanesini kullanır.
- İş parçacığı birleştirilebilir (joinable) ya da ayrık (detached) olabilir. Birleştirilebilir iş parçacığı `pthread_join` ile beklenir.
- Java'da iki yol: `Thread` sınıfından türetip `run()` metodunu yazmak ya da `Runnable` arayüzünü uygulamak. İkincisi daha yaygındır.
- İptal türleri: asenkron iptal (hedef hemen sonlandırılır) ve ertelenmiş iptal (deferred cancellation; hedef belli noktalarda kontrol edip kendini sonlandırır).
- TLS: her iş parçacığına özel veri alanı. Yerel değişkenlerle karıştırılmamalı.

Detaylı Açıklamalar

Neden iş parçacığı? İşlemciyi verimli kullanmak için birden fazla süreç çalıştırırız. Ama yeni bir süreç oluşturmak pahalıdır: kod, veri, yığın bellek ve yığın alanlarının hepsi kopyalanır. Oysa bir akışın ilerlemesi için temelde bir program sayacına, yazmaçlara ve bir yığına ihtiyaç vardır. İş parçacığı yalnızca bunları ayrı tutar, geri kalan her şeyi süreçle paylaşır. Böylece iki iş parçacıklı bir süreç, iki ayrı süreçten çok daha az bellek kullanır.

Web sunucusu örneği: Tek süreçli bir sunucuda istemciler kuyrukta bekler. Her istek için yeni süreç oluşturmak yavaştır ve kötüye kullanıma açıktır; binlerce istek binlerce süreç demektir. Her istek için bir iş parçacığı oluşturmak daha hızlı ve daha hafiftir.

Çok çekirdekli programlama. Bir işi paralel hale getirmenin iki temel yolu vardır: - **Veri paralelliği:** büyük veri parçalara bölünür, her parça ayrı çekirdekte işlenir. Örnek: Full HD bir

görüntü üzerinde 3×3 çekirdek (kernel) gezdirmek. Bölme yaparken komşu satırların bağımlılığı dikkate alınmalıdır. - **Görev paralelliği:** birbirinden bağımsız aktiviteler farklı iş parçacıklarına verilir. Örnek: metin düzenleyicide giriş alma, ekranı güncelleme ve yazım denetimi.

Çok iş parçacıklı programlarda hata bulmak zordur. Her iş parçacığı herhangi bir anda programın farklı bir yerinde olabilir. Olası durumların sayısı çok fazladır.

Amdahl Yasası. Programın S kadarı seri, $1-S$ kadarı paraleldir. Paralel kısım N çekirdeğe bölünür, seri kısım bölünemez:

$$\text{hızlanma} \leq \frac{1}{S + \frac{1-S}{N}}$$

Örnek: %25 seri bir programda çekirdek sayısını 1'den 2'ye çıkarmak hızı 2 kat değil, 1.6 kat artırır. Çekirdek sayısı çok büyürse paralel kısmın etkisi sıfıra yaklaşır. Hızlanma en çok $1/S$ olur.

Kullanıcı ve çekirdek iş parçacıkları. Kullanıcı seviyesinde iş parçacığı desteği, uygulamaya eklenen bir kütüphaneyle sağlanır. Kütüphanenin kodu ve veri yapıları kullanıcı alanındadır. Çekirdek seviyesinde ise kütüphanenin kodu ve veri yapıları çekirdektedir; kütüphane fonksiyonu çağrılınca bir sistem çağrısı yapılır. Günümüzde Windows, Linux, macOS ve Solaris çekirdek seviyesinde iş parçacığı destekler.

Pthreads örneği. Program komut satırından pozitif bir tam sayı alır. Bir iş parçacığı oluşturur ve bu iş parçacığı 1'den o sayıya kadar olan sayıları global `sum` değişkenine toplar. Ana iş parçacığı `pthread_join` ile bekler ve `sum`'ı yazar.

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

int sum;                    /* iş parçacıkları arasında ortak */
void *runner(void *param);

int main(int argc, char *argv[]) {
    pthread_t tid;
    pthread_attr_t attr;
    if (argc != 2 || atoi(argv[1]) < 0) return -1;
    pthread_attr_init(&attr);
    pthread_create(&tid, &attr, runner, argv[1]);
    pthread_join(tid, NULL);
    printf("sum = %d\n", sum);
}

void *runner(void *param) {
    int upper = atoi(param);
    sum = 0;
    for (int i = 1; i <= upper; i++) sum += i;
    pthread_exit(0);
}
```

- İş parçacığı fonksiyonu `void *` alır ve `void *` döndürür. Birden fazla parametre gerekiyorsa bir `struct` oluşturup adresi gönderilir.
- Global değişken bütün iş parçacıklarınca görülür. İkinci bir iş parçacığı aynı değişkene yazarsa sonuç bozulabilir. Bu yüzden erişim kontrollü yapılmalıdır.

Örtük iş parçacığı kullanımı. İş parçacıklarının oluşturulması ve yönetimi derleyiciye ya da çalışma zamanı kütüphanesine bırakılır. - İş parçacığı havuzu: önceden belli sayıda iş parçacığı oluşturulur, gelen işler bunlara verilir. - OpenMP: kodda `#pragma` yönergesiyle paralelleştirilecek bölge işaretlenir, gerisini derleyici yapar. - Grand Central Dispatch: macOS'ta kod blokları işaretlenir, çalışma zamanı kütüphanesi bunları paralel çalıştırır.

`fork()` ve `exec()`. Çok iş parçacıklı bir süreçte bir iş parçacığı `fork()` çağırırsa ne olmalı? Ya bütün iş parçacıkları kopyalanır ya da yalnızca çağıran iş parçacığı. Hangisinin doğru olduğu programcının amacına bağlıdır. `fork()`'tan hemen sonra `exec()` çağrılacaksa diğer iş parçacıklarını kopyalamak gereksizdir. Bu yüzden bazı UNIX sistemlerinde `fork()`'un iki sürümü vardır. `exec()` için ayrım gerekmez, çünkü süreç içeriğinin tamamı yeni programla değiştirilir.

Sinyaller. Çok iş parçacıklı süreçte sinyal hangi iş parçacığına gitmeli? Seçenekler: sinyalin ilgili olduğu iş parçacığına, bütün iş parçacıklarına, belli iş parçacıklarına ya da sinyalleri almakla görevli bir iş parçacığına. Sinyal birden fazla iş parçacığına giderse aynı işleyici (handler) birden fazla kez çalışabilir.

İş parçacığı iptali. Örnek: çok iş parçacıklı bir aramada bir iş parçacığı sonucu bulunca diğerlerini durdurmak isteriz. - Asenkron iptal: hedef iş parçacığı hemen sonlandırılır. Sorun, iş parçacığının kullandığı kaynakların düzgün geri verilmemesidir. - Ertelenmiş iptal: hedef iş parçacığı belli noktalarda iptal isteği var mı diye bakar ve varsa kendini düzgünce sonlandırır. Daha temizdir, ama istek ile sonlanma arasında gecikme olur.

Hafif süreç ve zamanlayıcı aktivasyonları. Çoktan çoka ve iki seviyeli modellerde kullanıcı kütüphanesi ile çekirdek arasında bir ara yapı gerekir: hafif süreç (LWP). Kütüphane LWP'yi sanal bir işlemci gibi görür ve kullanıcı iş parçacıklarını ona eşler. Her LWP bir çekirdek iş parçacığına bağlıdır. Çekirdek iş parçacığı bloklanırsa çekirdek bunu kütüphaneye bildirir (upcall) ve yeni bir LWP sağlar. Böylece bloklanma en aza iner.

Ders sonrası notlar

- Derste "Windows iş parçacıkları da kullanıcı seviyesinde sağlanıyor" dendi, sonra Windows kütüphanesinin çekirdek seviyesinde olduğu söylendi. Doğrusu ikincisi: Windows iş parçacığı kütüphanesi çekirdek seviyesindedir. Kullanıcı ya da çekirdek seviyesinde gerçeklenebilen kütüphane Pthreads'tir.
- "8 iş parçacığı destekli işlemcide genelde 4 çekirdek vardır" ifadesi Intel'in Hyper-Threading (SMT) teknolojisi için doğrudur. Ancak bu yalnızca zaman paylaşımı değildir. Her çekirdek iki donanım iş parçacığının komutlarını aynı anda yürütür ve çekirdeğin birimlerini paylaştırır.
- Amdahl Yasasında $N \to \infty$ iken "küçük bir sayıya gider" denen şey paralel terim $\frac{1-S}{N}$'dir. Hızlanmanın kendisi $\frac{1}{S}$'ye yaklaşır.
- Asenkron iptalde asıl sorun TLS'nin kalması değil, iş parçacığının elindeki kaynakların (tuttuğu kilitler, ayırdığı bellek gibi) geri verilmemesidir. Pthreads'te varsayılan iptal türü ertelenmiş iptaldir. İptal kontrolü iptal noktalarında (cancellation point, örneğin `pthread_testcancel()`) yapılır.
- Linux'ta ve POSIX'te `fork()` yalnızca çağıran iş parçacığını kopyalar. Bütün iş parçacıklarını kopyalayan sürüm bazı eski UNIX sistemlerinde (Solaris'teki `forkall()` gibi) bulunur.
- Amdahl hesabı (0.25 seri, $N = 2 \to 1.6$) Python ile doğrulandı.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.